

OccazCar

Application Mobile de Petites Annonces Automobiles

Rapport Technique

Contributeurs

Bechir ZAMMOURI
Chaima ABBES
Elyes SELLAMI

Projet Flutter - Firebase
4 janvier 2026

1 Introduction

OccazCar est une application mobile multiplateforme développée avec Flutter, destinée à faciliter l'achat et la vente de véhicules d'occasion. L'application propose une interface moderne et intuitive permettant aux vendeurs de publier leurs annonces et aux acheteurs de rechercher des véhicules selon différents critères.

2 Besoins Fonctionnels

2.1 Authentification et Gestion des Utilisateurs

- Inscription par email et mot de passe
- Connexion sécurisée avec Firebase Authentication
- Gestion du profil utilisateur
- Déconnexion

2.2 Fonctionnalités Vendeur

- Publication d'annonces avec informations détaillées (marque, modèle, année, kilométrage, prix)
- Upload multiple de photos via Firebase Storage
- Géolocalisation automatique du véhicule
- Modification des annonces existantes
- Suppression d'annonces
- Visualisation de toutes ses annonces

2.3 Fonctionnalités Acheteur

- Consultation de la liste complète des véhicules disponibles
- Recherche avancée avec filtres multiples (marque, prix maximum, année minimale)
- Visualisation détaillée d'un véhicule avec galerie photos
- Localisation GPS du véhicule sur carte interactive
- Contact direct du vendeur (appel téléphonique, SMS)

3 Besoins Non Fonctionnels

3.1 Performance

- Chargement optimisé des images avec mise en cache (`cached_network_image`)
- Temps de réponse inférieur à 2 secondes pour les opérations courantes
- Synchronisation en temps réel avec Firestore

3.2 Sécurité

- Authentification sécurisée via Firebase Authentication
- Règles de sécurité Firestore pour protéger les données
- Isolation des données utilisateurs
- Validation côté client et serveur

3.3 Disponibilité et Fiabilité

- Infrastructure Firebase Cloud avec haute disponibilité (99.95%)
- Gestion des erreurs et feedback utilisateur
- Mode hors ligne partiel avec cache local

3.4 Compatibilité

- Support Android et iOS
- SDK Flutter minimum : 3.0.6
- Design responsive et adaptatif
- Material Design 3 pour une expérience utilisateur moderne

3.5 Maintenabilité

- Architecture modulaire avec séparation des responsabilités
- Code organisé en couches (Models, Services, Screens)
- Gestion d'état avec Provider
- Documentation du code

4 Architecture Logicielle

4.1 Architecture Logique

L'application adopte une **architecture en couches** inspirée du pattern **MVVM** (Model-View-ViewModel) via Provider :

Couches de l'Application

- Présentation** Interfaces utilisateur (Screens) développées avec Flutter Widgets
- Logique Métier** Services et Providers pour la gestion d'état et logique applicative
- Données** Modèles de données et accès aux sources externes (Firebase)
- Infrastructure** Services Firebase (Auth, Firestore, Storage)

4.1.1 Structure des Composants

1. Couche Modèles (Models)

- **UserModel** : Représentation des utilisateurs
- **VehicleModel** : Entité véhicule avec 15 attributs (id, sellerId, brand, model, year, mileage, price, images, GPS, etc.)

2. Couche Services

- **AuthService** : Gestion authentification et état utilisateur avec Provider
- **DatabaseService** : CRUD véhicules, requêtes Firestore, recherche filtrée
- **StorageService** : Upload/suppression images Firebase Storage

3. Couche Présentation (Screens)

- **Auth** : LoginScreen, RegisterScreen

- **Buyer** : VehicleListScreen, VehicleDetailScreen, ModernSearchScreen
- **Seller** : AddVehicleScreen, EditVehicleScreen, MyVehiclesScreen
- **Profile** : Gestion profil utilisateur
- **Home** : Navigation principale

4. Utilitaires

- **AppTheme** : Thème Material Design 3 centralisé
- **AppColors** : Palette de couleurs cohérente

4.2 Architecture Physique

Infrastructure Déployée

Architecture Client-Serveur Cloud

4.2.1 Composants Physiques

1. Client Mobile (Flutter)

- Application compilée nativement pour Android (ARM/x86) et iOS (ARM64)
- Stockage local : Cache images, préférences utilisateur (SharedPreferences)
- SDK Flutter avec Dart VM

2. Backend Firebase (Google Cloud)

- **Firebase Authentication** : Serveurs d'authentification distribués
- **Cloud Firestore** : Base de données NoSQL distribuée
 - Collection **users** : Profils utilisateurs
 - Collection **vehicles** : Annonces véhicules
- **Firebase Storage** : Stockage objet pour images (buckets régionaux)

3. Services Tiers

- **Google Maps API** : Géolocalisation et cartographie
- **Geolocator** : Récupération position GPS du device

4.2.2 Flux de Communication

1. **Authentification** : App ↔ Firebase Auth (HTTPS/REST)
2. **Données temps réel** : App ↔ Firestore (WebSocket via gRPC)
3. **Images** : App ↔ Firebase Storage (HTTPS)
4. **GPS** : App ↔ Plateforme native ↔ Google Maps API

5 Technologies Utilisées

5.1 Frontend

Flutter 3.0.6+ (Dart 3.0.6+)

- Framework UI multiplateforme de Google
- Compilation native (AOT) pour performance optimale
- Hot Reload pour développement rapide
- Widgets Material Design 3

5.2 Backend et Services

`firebase_core` Initialisation SDK Firebase
`firebase_auth` Authentification utilisateurs (email/password)
`cloud_firestore` Base de données NoSQL temps réel
`firebase_storage` Stockage cloud pour images

5.3 Gestion d'État et Navigation

`provider` Pattern Observer pour gestion d'état réactive

5.4 Fonctionnalités Métier

`image_picker` Sélection photos depuis galerie/caméra
`geolocator` Géolocalisation GPS du device
`url_launcher` Lancement appels/SMS vers vendeurs
`cached_network_image` Cache intelligent images réseau
`intl` Internationalisation et formatage dates/nombres

5.5 Outils de Développement

`flutter_lints` Règles de qualité code Dart
`flutter_test` Framework de tests unitaires et widgets

5.6 Infrastructure

- **Google Cloud Platform** : Hébergement Firebase
- **Android Studio** / **Xcode** : Compilation native
- **Git** : Contrôle de version

6 Modèle de Données

6.1 Collection Firestore : vehicles

Champ	Type	Description
id	String	Identifiant unique (auto-généré)
sellerId	String	ID utilisateur vendeur
sellerName	String	Nom du vendeur
sellerPhone	String	Téléphone du vendeur
brand	String	Marque du véhicule
model	String	Modèle du véhicule
year	Integer	Année de fabrication
mileage	Integer	Kilométrage
price	Double	Prix en devise locale
description	String	Description détaillée
images	Array<String>	URLs des photos
latitude	Double	Coordonnée GPS
longitude	Double	Coordonnée GPS
location	String	Adresse textuelle
createdAt	Timestamp	Date de création
status	String	Statut (active/sold/deleted)

7 Conclusion

OccazCar est une solution moderne et complète pour la gestion de petites annonces automobiles. L'architecture choisie offre :

- **Scalabilité** : Infrastructure Firebase cloud élastique
- **Maintenabilité** : Code modulaire et bien structuré
- **Performance** : Compilation native et optimisations
- **Expérience utilisateur** : Interface fluide et responsive

L'utilisation de Flutter et Firebase permet un développement rapide tout en garantissant une application professionnelle et performante sur Android et iOS.